

HPPS Assignment A

Björn Petterson (zts900), Thomas Andersen (kgq325)

April 15, 2026

Introduction

We are largely satisfied by our implementations. The restrictions on control flow has challenged the way we implemented some functions, as some seem to lend themselves to using loops and if-statements. We suspect that implementation would have been less dense using a `for-loop` but we still feel that we have reached implementations using reasonable and readable code.

While we did implement both `get_bit` and `set_bit` we ended up only using one of them, which does hint that there might be some insight to be gained.

In an ideal world errors and exceptions would be handled by the functions. This includes but it not limited to type testing arguments passed to the functions. This means that we can reasonably easily brake the functions during testing if we wanted/needed to. What we have done instead, which we believe is in line with the scope of the assignment, is to test functions "lightly". One of the consequences of this is that we have generally steered clear of testing that would cause overflow.

Testing in `test_numbers.c` is implemented using `assert` with different test values. To run the suite of tests run `make test_numbers` from the terminal.

Task 2: Converting to and from C integers

To implement `bits8_from_int` we decided to implement the suggested helper function `get_bit`. Besides the helper function we make use of `bit_from_int` from `bits.h` for each bit in the bit vector individually.

For `bits8_to_int` we use `bit_to_int` to convert each bit to an int and then left shift for multiplication. We keep track of the sum in an unsigned int `y` using bit-wise OR when

updating y at each step.

Task 3: Implementing addition

For the implementation of `bits8_add` it was suggested that we implement a function for the addition and a `struct` to hold results. We have decided to use two `struct bits8` to hold values for sums and carries plus and a separate overflow bit for the last carry. After re-reading the task description as well as the hints we see that there is some benefit to constructing a new `struct` to hold both sum and carry. However, as the prototype indicates we are not to return both the sum and 9th bit carry. Therefore we reasoned it was still a decent implementation given the restrictions.

Task 4: Implementing negation

For this implementation we just use `bit_not` from `bits.h`, storing the result and incrementing by 1 using `bits8_add` as suggested.

An interesting part of this function/task is how it illustrates how much interpretation impacts what a function does in cases such as this. If we were to continue to interpret the bit vector as unsigned this function no longer produce the desired result of multiplying by -1.

Another interesting point comes up during testing. If using `assert` to compare results of using the function on a negative number the resulting number is positive and will therefore compare favorably. However, if we start with a positive number we get a warning. This happens as the conversion back to integer does result in a negative integer.

Task 5: Implementing Multiplication

The multiplication function was implemented following the steps below:

Steps to Implement:

1. For every digit y_i in y (starting from the right):
 - (a) For every digit x_j in x (starting from the right):
 - i. Multiply $x_j \cdot y_i$.

- ii. Note the result down under y , from right to left.
 - iii. Shift the expression i -steps to the left.
- (b) This results in a number.
2. This process results in i numbers, each shifted progressively one step to the left.
 3. Add the resulting numbers together using the standard high-school addition algorithm.
 4. This results in one final number.

Implementation:

Key here is that we need to implement this without standard C functionality like loops or `if-else` statements. To avoid excessive lines of code and to make the implementation more readable and manageable, we create a helper function to handle the multiplication of one bit with one 8-bit number, corresponding to step 1.1.1.

This helper function takes in the x_i bit as a boolean (`b`) and y , and returns y if $x_i = 1$, or a zero 8-bit vector if $x_i = 0$. This is achieved using 8 lines of `&&` operations: for every y_j bit in y , the logic evaluates $y_j \& b$.

We save the result of every 1.1.1 step as an 8-bit number that we progressively shift to the left.

Finally, we add all the shifted bits together and return the result.

Task-specific questions

Does `bits8_add()` provide “correct” results if you pass in negative numbers in two’s complement representation? Why or why not?

Yes it does. The reason is because addition is handled in the same way for two’s complement as it would for an unsigned integer. If we consider -1 represented in two’s complement would be `<11111111>`. As an unsigned integer it would represent 255. If we add 1 `<00000001>` we would cause overflow and the resulting bit vector would be `<00000000>` which is 0 in both representations.

Does `bits8_mul()` provide “correct” results if you pass in negative numbers in two’s complement representation?

Yes - because the mathematical operators used to do the calculation, AND and SHIFT, are still compatible with using 2s complement. If you look at the function you will see these are the only operations I use. In essence two’s complement does not behave differently mathematically than an unsigned int, the difference is in interpretation rather than mathematical properties. Hence all the calculations addition subtraction multiplication division etc that I can do on an unsigned int I can do on two’s complement without changing the relationship between the factors and product in a multiplication operation, only the interpretation changes.

For comparison I will translate these operations to base 10, but keep in mind that in our 8 bit world we have a modular timeline that wraps around 256 rather than continuing forever like in base 10, hence I use mod 256 to see whether both these operations are actually mathematically viable, ofc this is not a complete proof but an example.

Example shift operation: $11111110 \ll 2 = 11111100$

For unsigned int: In decimal we have $(254 * 2) \bmod 256 = 252$.

For two’s complement: In decimal we have $(-2 * 2) \bmod 256 = -4$.

Example and operation: $11001101 \& 00001111 = 00001101$.

For unsigned int: $11001101 = 205$ $00001111 = 15$ $00001101 = 13$

For two’s complement: $11001101 = -51$ $00001111 = 15$ $00001101 = 13$

In other words we see that even though the interpretation of 2s complement vs 8 bit ints are different applying these operations does not break the underlying logic.

I remember groups being mentioned very briefly in another course at some point in relation to mathematical objects and operations, I conversed with chat gpt about this and got the answer that ring is a more accurate term specifically modular arithmetic rings closed under addition subtraction and multiplication modulo 2^n . It makes sense that there would be some mathematical description that concludes that both interpretations are part of a system with some formal name "group" or "ring" on which we can apply the same operations without breaking it whether 2s complement or unsigned int, however I don’t know enough to say whether this exact description is accurate.

How would you implement a function `bits8_sub()` for subtracting 8-bit numbers?

By making sure that the function handles 8-bit unsigned integers, taking into consideration the possible overflow/ underflow that can happen.

I would use a header `<stdint.h>` that can allow me to use a `uint8_t` type, which assures me that both input and output are 8-bit integers. If the case was a need of negative results, i would rather use `int8_t`

Overall i would first make sure that the inputs are in the 8-bit range for respectively signed or unsigned values, by using the `uint8_t` type. Therefore it would lead to a function that subtracts two 8-bit unsigned numbers.

I would then make an if statements that checks for underflow if there occurs and borrow (if $b > a$).

If thats the case, then it would simulate a modular subtraction.

The following else statement will then have borrow equal 0, because its not needed, (due to b not being greater than a), and it would then perform a direct subtraction.

No matter if there is underflow or not, it would return the result of the subtraction in an 8 bit value.